Git & GitHub — Complete Workflow Notes

Beginner → Company-style Branching, Commit, Push & Pull Request Workflow

1. Git aur GitHub kya hain?

Git local computer par code ki version history manage karta hai. **GitHub** remote/server par Git repositories host karta hai aur collaboration, Pull Requests, review aur merging provide karta hai.

2. Repository clone karna

GitHub repository ko local computer par lane ke liye:

```
git clone https://github.com/Arham38/task.git
```

Clone ke baad repository folder ke andar jana zaroori hai:

```
cd task
git status
```

Agar aap repository ke bahar ho aur *not a git repository* aaye, pehle correct folder mein **cd** karo.

3. Main ko production banana

```
git branch -m main production
git push -u origin production
```

Yahan local **main** ko **production** rename karke remote par push kiya gaya.

4. Dev branch banana

```
git switch -c dev
git push -u origin dev
```

dev ko testing/integration branch ki tarah use kiya ja sakta hai. Har company ka exact workflow different ho sakta hai.

5. Feature branches

Har new task/feature ke liye dev se separate branch banana clean workflow hai:

```
git switch dev
git switch -c feature/add_task
```

Example naming: **feature/add_task**, **feature/update_task**.

6. Java project example

Practice mein Task Manager banaya gaya tha. Files:

```
Main.java
Task.java
TaskManager.java
```

Java compile/run:

```
javac *.java
java Main
```

Important: public class ka filename match hona chahiye. Example: **public class TaskManager** → file **TaskManager.java**.

7. Git status → add → commit → push

```
git status
git add .
git commit -m "Simple Task Manager code written"
git push -u origin feature/add_task
```

Flow: working files → staging area → commit → remote GitHub.

8. Wrong branch name ki common mistake

```
git push -u origin feature/task
```

Agar actual branch **feature/add_task** ho to *src refspec ... does not match any* aa sakta hai. Correct branch name check karne ke liye:

```
git branch
```

9. .class files aur .gitignore

Java compile karne par .class files banti hain. Normally in generated files ko repository mein commit nahi karna chahiye.

```
*.class
```

Is line ko **.gitignore** mein rakh sakte hain. Agar .class pehle se tracked ho chuki hon:

```
git rm --cached *.class
git add .
git commit -m "Ignore compiled Java files"
git push
```

10. Pull Request — Base vs Compare

Base = destination — code kahan jaana hai.
Compare = source — code kis branch se aa raha hai.

Purpose	Base	Compare
Feature ko testing mein bhejna	dev	feature/add_task
Release ko production mein bhejna	production	dev

Easy rule: **Base = jahan code jaana hai.**

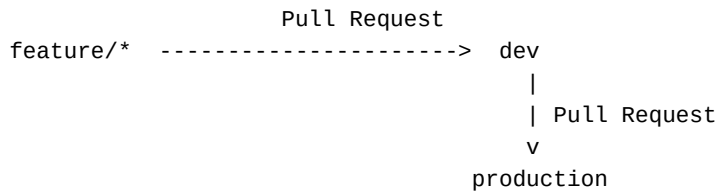
11. Accidental PR merge — kya hua?

Practice mein feature/add_task ko galti se **production** mein merge kar diya gaya, jab intended workflow feature → dev tha. Is se feature testing branch ko bypass karke production mein chala gaya.

Is situation mein practice repository ko sync karne ke liye production ke latest changes dev mein laaye gaye:

```
git switch production
git pull origin production
git switch dev
git merge production
git push origin dev
```

12. Current company-style workflow



Typical process: task → feature branch → coding → commit → push → PR to dev → review/approval → merge into dev → testing/integration → PR dev to production → release.

13. Useful daily commands

Command	Purpose
git status	Current changes aur branch status dekhna
git branch	Local branches dekhna
git switch <branch>	Existing branch par switch karna
git switch -c <branch>	New branch create + switch
git add .	Changes staging mein add karna
git commit -m "message"	Snapshot/commit banana
git push	Local commits remote par bhejna
git pull origin <branch>	Remote changes local branch mein lana
git merge <branch>	Dusri branch ke changes current branch mein merge karna
git log --oneline	Short commit history dekhna

14. Common mistakes checklist

• Repository ke bahar git status chalana → pehle cd task.

• Wrong branch name push karna → git branch se exact naam check karo.

• PR mein base galat select karna → Base destination hota hai.

• .class files commit karna → .gitignore mein *.class.

• Feature ka kaam directly production par karna → feature branch use karo.

• Remote changes ignore karna → zarurat par git pull.

15. Practical workflow — commands

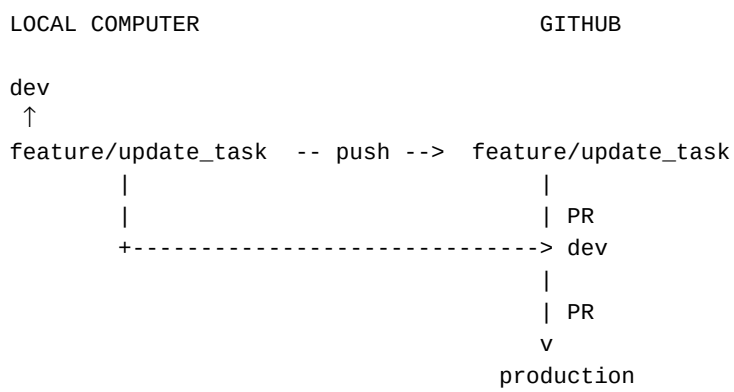
```
# Repository
git clone <repo-url>
cd task

# Branch setup
git branch -m main production
git push -u origin production
git switch -c dev
git push -u origin dev

# New feature
git switch dev
git switch -c feature/update_task

# Work + save
git status
git add .
git commit -m "Update task feature"
git push -u origin feature/update_task

# GitHub
PR: base = dev
    compare = feature/update_task

# After merge into dev
git switch dev
git pull origin dev
```

```
# Release
PR: base = production
    compare = dev
```

16. Learning checkpoint

Practice quiz mein 7 questions answer kiye gaye: 6 correct, 1 incorrect. Incorrect concept: **git clone** ke baad Git commands har parent folder se automatically repository ke andar execute nahi hoti; repository ke correct directory context mein hona zaroori hai.

17. Final mental model



Golden rule: Feature branch par kaam karo. Feature ka PR normally **dev** ko. Release ke waqt **dev** → **production** PR.